

Probable Suggestive Question for Object Oriented Programming

1. What are the key differences between Java and C++?
2. What is the role of JRE (Java Runtime Environment) in the Java ecosystem?
3. Can an abstract class have a constructor in Java? Justify your answer.
4. List and explain any two access specifiers in Java with examples.
5. What is inheritance? Provide a real-world example.
6. Differentiate between method overloading and method overriding.
7. Define class and object in Java with one simple example.
8. Explain the difference between break and continue statements in Java.
9. Can we override a private method in Java? Why or why not?
10. What is the use of the super keyword in Java?
11. Define polymorphism and give one code example.
12. What is the difference between ArrayList and an array in Java?
13. Differentiate between stack memory and heap memory in Java.
14. What is constructor overloading? Explain with an example.
15. What is the role of static keyword in Java?
16. Explain the difference between String, StringBuilder, and StringBuffer.
17. What is garbage collection in Java? How does it work?
18. What are wrapper classes in Java? Why are they needed?
19. Write a Java program to define a class Student with attributes name, rollNo, and marks. Include a method to display student details and create an object to invoke the method.
20. Write a Java program to create an abstract class Vehicle with an abstract method move(). Derive two classes Bike and Car that override the method and demonstrate runtime polymorphism.
21. Write a Java program to illustrate the use of the this() keyword to call another constructor in the same class.
22. Write a Java program to define a class BankAccount with deposit and withdraw methods. Create an object and demonstrate the functionality.
23. Write a Java program to demonstrate method overloading using a class Calculator with methods add(int, int) and add(double, double).
24. Write a Java program to show the use of final keyword with variable, method, and class.

25. Write a Java program to demonstrate user input using Scanner class and print student details like name, age, and grade.
26. Write a Java program using an interface Printable with a method print(). Create a class Document that implements the interface and defines the method.
27. Write a Java program to demonstrate multiple constructors in a class Book using constructor overloading.
28. Write a Java program to demonstrate **multilevel inheritance** using Animal → Mammal → Dog hierarchy. Show method calling at each level.
29. Write a Java program to implement **multiple inheritance using interfaces** (Flyable and Swimmable) with a class Duck.
30. Write a Java program to create a base class Shape with a method draw(). Create two subclasses Triangle and Square that override the draw() method. Demonstrate runtime polymorphism.
31. Write a Java program to accept a sentence from the user and count the number of vowels and consonants using String methods.
32. Write a Java program to use ArrayList to store the names of students. Perform the following operations: add five names, display them, remove one, and print the updated list.
33. Write a Java program to implement a simple student grading system using classes and arrays of objects.

Solution

1. What are the key differences between Java and C++?

Feature	Java	C++
Platform	Platform-independent (WORA)	Platform-dependent
Memory Management	Automatic (Garbage Collector)	Manual (new/delete)
Multiple Inheritance	Via interfaces	Directly supported
Pointers	No direct pointer support	Supports pointers
Compilation	Bytecode (JVM)	Native Machine Code

2. What is the role of JRE (Java Runtime Environment)?

JRE provides the **environment** required to run Java applications. It contains:

- JVM (Java Virtual Machine)

- Java class libraries
- Java class loader

Without JRE, Java programs cannot be executed.

3. Can an abstract class have a constructor in Java?

Yes. An abstract class **can have a constructor**. It is used to initialize common properties or perform setup for the subclass.

```
abstract class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
}
```

4. List and explain any two access specifiers in Java.

1. public: Accessible from **anywhere**.
2. private: Accessible **only within the class** where declared.

5. What is inheritance? Give a real-world example.

Inheritance is when one class derives properties and behavior from another class.

 *Example:*

A Dog class can inherit from an Animal class.

6. Differentiate between method overloading and method overriding.

Feature	Overloading	Overriding
Same method name	✓	✓
Parameters	Different	Same
Class	Same class	Parent & child class
Compile time or runtime	Compile-time	Runtime

7. Define class and object with a simple example.

- **Class:** Blueprint for creating objects

- **Object:** Instance of the class

```
class Car {  
    String brand = "Toyota";  
}  
  
Car c = new Car();
```

8. Difference between break and continue:

- break: Exits the loop entirely.
 - continue: Skips to the next iteration of the loop.
-

9. Can we override a private method in Java?

No. Private methods are **not inherited** by subclasses, hence cannot be overridden.

10. Use of super keyword:

Used to access:

- Superclass constructor
- Superclass method
- Superclass variable

```
super.display();
```

11. Define polymorphism with example.

Polymorphism: Same method behaves differently based on context.

```
class Animal {  
    void sound() { System.out.println("Animal sound"); }  
}  
  
class Dog extends Animal {  
    void sound() { System.out.println("Bark"); }  
}
```

12. Difference between Array and ArrayList:

Feature	Array	ArrayList
Size	Fixed	Dynamic
Type	Can store primitives	Stores objects only
Methods	No utility methods	Many built-in methods

13. Stack vs Heap memory:

- **Stack:** Stores function calls, local variables. Faster.
 - **Heap:** Stores objects. Larger but slower.
-

14. Constructor overloading:

Creating multiple constructors with different parameter lists.

```
class Book {  
    Book() {}  
    Book(String title) {}  
}
```

15. Role of static keyword:

Used to create:

- Static variables (shared)
 - Static methods
 - Static blocks
 - Static nested classes
-

16. String vs StringBuilder vs StringBuffer:

Feature	String	StringBuilder	StringBuffer
Mutable	✗	✓	✓
Thread-safe	✗	✗	✓
Speed	Slow	Fast	Slower (due to sync)

17. What is garbage collection?

Automatic memory management. JVM reclaims memory used by **unreferenced objects**.

18. What are wrapper classes?

They convert **primitive types** into **objects** (boxing), e.g., int → Integer.

Used in:

- Collections (which store objects)
 - Serialization
 - Java generics
-

PROGRAMMING QUESTIONS & SOLUTIONS

1. Student class (attributes + display):

```
class Student {
    String name;
    int rollNo;
    double marks;

    void display() {
        System.out.println(name + " | Roll: " + rollNo + " | Marks: " + marks);
    }

    public static void main(String[] args) {
        Student s = new Student();
        s.name = "Alice";
        s.rollNo = 101;
        s.marks = 92.5;
        s.display();
    }
}
```

2. Abstract class Vehicle + Polymorphism:

```
abstract class Vehicle {
    abstract void move();
}

class Bike extends Vehicle {
    void move() { System.out.println("Bike moves on 2 wheels"); }
}
```

```

class Car extends Vehicle {
    void move() { System.out.println("Car moves on 4 wheels"); }
}

public class Test {
    public static void main(String[] args) {
        Vehicle v = new Bike(); v.move();
        v = new Car(); v.move();
    }
}

```

3. this() constructor chaining:

```

class Demo {
    Demo() {
        this(10);
        System.out.println("Default constructor");
    }
    Demo(int x) {
        System.out.println("Parameterized constructor: " + x);
    }

    public static void main(String[] args) {
        new Demo();
    }
}

```

4. BankAccount with deposit and withdraw:

```

class BankAccount {
    double balance = 0;

    void deposit(double amount) {
        balance += amount;
    }

    void withdraw(double amount) {
        if (amount <= balance)
            balance -= amount;
        else
            System.out.println("Insufficient balance");
    }

    void showBalance() {
        System.out.println("Balance: " + balance);
    }
}

```

```
public static void main(String[] args) {  
    BankAccount acc = new BankAccount();  
    acc.deposit(5000);  
    acc.withdraw(2000);  
    acc.showBalance();  
}  
}
```

5. Method Overloading (Calculator):

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        System.out.println(c.add(5, 3));  
        System.out.println(c.add(2.5, 3.7));  
    }  
}
```

6. final keyword demo:

```
final class A {  
    final int x = 10;  
  
    final void show() {  
        System.out.println("Final method");  
    }  
}
```

7. Scanner input (Student details):

```
import java.util.Scanner;  
  
class StudentInput {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```



```
System.out.print("Name: ");
String name = sc.nextLine();
System.out.print("Age: ");
int age = sc.nextInt();
System.out.print("Grade: ");
String grade = sc.next();

System.out.println("Student: " + name + " | Age: " + age + " | Grade: " + grade);
}
}
```

8. Interface Printable (Document):

```
interface Printable {
    void print();
}

class Document implements Printable {
    public void print() {
        System.out.println("Printing Document...");
    }

    public static void main(String[] args) {
        Printable p = new Document();
        p.print();
    }
}
```

9. Book with constructor overloading:

```
class Book {
    Book() {
        System.out.println("Default constructor");
    }

    Book(String title) {
        System.out.println("Title: " + title);
    }

    public static void main(String[] args) {
        new Book();
        new Book("OOP in Java");
    }
}
```

10. Multilevel Inheritance:

```
class Animal {
    void eat() { System.out.println("Animal eats"); }
}

class Mammal extends Animal {
    void walk() { System.out.println("Mammal walks"); }
}

class Dog extends Mammal {
    void bark() { System.out.println("Dog barks"); }

    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); d.walk(); d.bark();
    }
}
```

11. Multiple Inheritance via Interface (Duck):

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("Duck flies"); }
    public void swim() { System.out.println("Duck swims"); }

    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly(); d.swim();
    }
}
```

12. Shape class with draw() method (Polymorphism):

```
class Shape {
    void draw() {
        System.out.println("Drawing shape");
    }
}

class Triangle extends Shape {
    void draw() {
        System.out.println("Drawing triangle");
    }
}

class Square extends Shape {
    void draw() {
        System.out.println("Drawing square");
    }

    public static void main(String[] args) {
        Shape s = new Triangle(); s.draw();
        s = new Square(); s.draw();
    }
}
```

13. Count vowels/consonants using String:

```
import java.util.Scanner;

class VowelCounter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String input = sc.nextLine().toLowerCase();

        int v = 0, c = 0;
        for (char ch : input.toCharArray()) {
            if ("aeiou".indexOf(ch) != -1) v++;
            else if (ch >= 'a' && ch <= 'z') c++;
        }

        System.out.println("Vowels: " + v + ", Consonants: " + c);
    }
}
```

14. ArrayList - Student Names:

```
import java.util.*;

class NameList {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Alice"); names.add("Bob"); names.add("Cathy");
        names.add("David"); names.add("Eva");

        System.out.println("All names: " + names);
        names.remove("David");
        System.out.println("Updated: " + names);
    }
}
```

15. Student Grading System (Array of objects):

```
class Student {
    String name;
    int marks;

    void display() {
        System.out.println(name + " scored " + marks);
    }
}

public class GradingSystem {
    public static void main(String[] args) {
        Student[] students = new Student[3];
        for (int i = 0; i < 3; i++) {
            students[i] = new Student();
            students[i].name = "Student" + (i+1);
            students[i].marks = 70 + i * 5;
        }

        for (Student s : students) {
            s.display();
        }
    }
}
```

Q: Write a Java program to accept a sentence from the user and count the number of vowels and consonants using String methods.

```
import java.util.Scanner;

public class VowelConsonantCounter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter a sentence:");
        String input = scanner.nextLine().toLowerCase();

        int vowels = 0, consonants = 0;

        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);
            if (Character.isLetter(ch)) {
                if ("aeiou".indexOf(ch) != -1) vowels++;
                else consonants++;
            }
        }

        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);
    }
}
```

Q: Write a Java program to use ArrayList to store the names of students. Perform the following operations:

- Add five names
- Display all names
- Remove one name
- Print the updated list

```
import java.util.ArrayList;

public class StudentNameList {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();

        // Add 5 names
        students.add("Amit");
        students.add("Beena");
        students.add("Chirag");
        students.add("Deepa");
        students.add("Esha");

        // Display all names
        System.out.println("All Students: " + students);
    }
}
```

```
// Remove one name
students.remove("Chirag");

// Updated list
System.out.println("After Removal: " + students);
}
}
```

Q: Write a Java program to implement a simple student grading system using classes and arrays of objects.

```
class Student {
    String name;
    int marks;

    String getGrade() {
        if (marks >= 90) return "A";
        else if (marks >= 75) return "B";
        else if (marks >= 60) return "C";
        else return "D";
    }

    void display() {
        System.out.println(name + " - Marks: " + marks + " - Grade: " + getGrade());
    }
}

public class GradingSystem {
    public static void main(String[] args) {
        Student[] students = new Student[3];

        // Create sample data
        students[0] = new Student(); students[0].name = "Raj"; students[0].marks = 88;
        students[1] = new Student(); students[1].name = "Priya"; students[1].marks = 95;
        students[2] = new Student(); students[2].name = "Ali"; students[2].marks = 59;

        // Display all students
        for (Student s : students) {
            s.display();
        }
    }
}
```